

TP_Proposiciones_Compuestas\clasificador_proposiciones.py

```

1  # Clasificador de Proposiciones Compuestas
2  # Trabajo Integrador - Matemática y Programación I - UTN 2026
3  #
4  # Genera la tabla de verdad de una proposición lógica y la clasifica
5  # como TAUTOLOGÍA, CONTRADICCIÓN o CONTINGENCIA.
6
7  from itertools import product
8
9
10 # — 1. SEPARAR LA PROPOSICIÓN EN SÍMBOLOS —————
11 # Recorre el texto y arma una lista con cada parte por separado.
12 # Ejemplo: "p -> ~q" → ['p', '->', '~', 'q']
13
14 def separar_simbolos(proposicion):
15     simbolos = []
16     i = 0
17     while i < len(proposicion):
18         if proposicion[i] == ' ':
19             i += 1
20         elif proposicion[i:i+3] == '<->':      # bicondicional (3 caracteres)
21             simbolos.append('<->'); i += 3
22         elif proposicion[i:i+2] == '->':      # condicional (2 caracteres)
23             simbolos.append('->'); i += 2
24         elif proposicion[i] in '~&|()':
25             simbolos.append(proposicion[i]); i += 1
26         elif proposicion[i].isalpha():
27             simbolos.append(proposicion[i].lower()); i += 1
28         else:
29             raise ValueError(f"Símbolo no reconocido: '{proposicion[i]}'")
30     return simbolos
31
32
33 # — 2. EVALUAR LA PROPOSICIÓN —————
34 # Calcula el resultado respetando la precedencia matemática de operadores:
35 #
36 # <-> < -> < | < & < ~      (de menor a mayor precedencia)
37 #
38 # Cada función maneja un operador y llama a la del siguiente nivel.
39 # La variable 'pos' indica qué símbolo de la lista se está leyendo.
40
41 def eval_bicondicional(simbolos, pos, valores):
42     izq, pos = eval_condicional(simbolos, pos, valores)
43     while pos < len(simbolos) and simbolos[pos] == '<->':
44         pos += 1
45         der, pos = eval_condicional(simbolos, pos, valores)
46         izq = (izq == der)          # p <-> q: verdadero cuando ambos tienen el mismo valor
47     return izq, pos
48

```

```
49 def eval_condicional(simbolos, pos, valores):
50     izq, pos = eval_disyuncion(simbolos, pos, valores)
51     while pos < len(simbolos) and simbolos[pos] == '->':
52         pos += 1
53         der, pos = eval_disyuncion(simbolos, pos, valores)
54         izq = (not izq) or der      # p -> q equivale a ~p | q (falso solo si p=V y q=F)
55     return izq, pos
56
57 def eval_disyuncion(simbolos, pos, valores):
58     izq, pos = eval_conjuncion(simbolos, pos, valores)
59     while pos < len(simbolos) and simbolos[pos] == '|':
60         pos += 1
61         der, pos = eval_conjuncion(simbolos, pos, valores)
62         izq = izq or der
63     return izq, pos
64
65 def eval_conjuncion(simbolos, pos, valores):
66     izq, pos = eval_negacion(simbolos, pos, valores)
67     while pos < len(simbolos) and simbolos[pos] == '&':
68         pos += 1
69         der, pos = eval_negacion(simbolos, pos, valores)
70         izq = izq and der
71     return izq, pos
72
73 def eval_negacion(simbolos, pos, valores):
74     if pos < len(simbolos) and simbolos[pos] == '~':
75         pos += 1
76         resultado, pos = eval_negacion(simbolos, pos, valores) # permite ~~p
77         return not resultado, pos
78     return eval_variable(simbolos, pos, valores)
79
80 def eval_variable(simbolos, pos, valores):
81     if pos >= len(simbolos):
82         raise ValueError("Proposición incompleta.")
83     if simbolos[pos] == '(':
84         pos += 1
85         resultado, pos = eval_bicondicional(simbolos, pos, valores)
86         if pos >= len(simbolos) or simbolos[pos] != ')':
87             raise ValueError("Falta cerrar paréntesis.")
88         return resultado, pos + 1
89     if simbolos[pos] in valores:
90         return valores[simbolos[pos]], pos + 1
91     raise ValueError(f"Variable no reconocida: '{simbolos[pos]}'")
92
93 def evaluar(proposicion, valores):
94     simbolos = separar_simbolos(proposicion)
95     resultado, _ = eval_bicondicional(simbolos, 0, valores)
96     return resultado
97
98
```

```
99 # — 3. TABLA DE VERDAD Y CLASIFICACIÓN
100
101 def generar_tabla(proposicion):
102     # Extrae las variables únicas y genera las 2^n combinaciones posibles
103     variables = sorted(set(c for c in proposicion.lower() if c.isalpha()))
104     if not variables:
105         raise ValueError("La proposición no contiene variables.")
106     filas = []
107     for combinacion in product([False, True], repeat=len(variables)):
108         valores = dict(zip(variables, combinacion))
109         filas.append({'valores': valores, 'resultado': evaluar(proposicion, valores)})
110     return variables, filas
111
112 def clasificar(filas):
113     resultados = [f['resultado'] for f in filas]
114     if all(resultados): return "TAUTOLOGÍA"
115     if not any(resultados): return "CONTRADICCIÓN"
116     return "CONTINGENCIA"
117
118
119 # — 4. MOSTRAR RESULTADOS
120
121 def mostrar_tabla(proposicion, variables, filas, clasificacion):
122     encabezado = " " + " | ".join(f" {v} " for v in variables) + " | Resultado"
123     separador = " " + "-" * (len(encabezado) - 2)
124
125     print(f"\n{'='*55}")
126     print(f" Proposición: {proposicion}")
127     print(f"{'='*55}")
128     print(encabezado)
129     print(separador)
130     for fila in filas:
131         celdas = " | ".join(f" {v} " if fila['valores'][v] else 'F' for v in variables)
132         res = 'V' if fila['resultado'] else 'F'
133         print(f" {celdas} | {res}")
134     print(separador)
135
136     print(f"\n >>> {clasificacion} <<<\n")
137     if clasificacion == "TAUTOLOGÍA":
138         print(" Verdadera en TODOS los casos.")
139         print(" El resultado no depende de los valores de las variables.")
140     elif clasificacion == "CONTRADICCIÓN":
141         print(" Falsa en TODOS los casos.")
142         print(" El resultado no depende de los valores de las variables.")
143     else:
144         verdaderas = sum(1 for f in filas if f['resultado'])
145         print(f" Verdadera en {verdaderas} de {len(filas)} casos.")
146         print(" El resultado depende de los valores de las variables.")
147     print("=" * 55)
148
```

```
149
150 # — 5. MENÚ —————
151
152 EJEMPLOS = [
153     ("p & ~p", "Contradicción clásica"),
154     ("p | ~p", "Tautología (Tercio Excluido)",
155     ("p -> q", "Condicional simple"),
156     ("(p -> q) <-> (~q -> ~p)", "Ley de la Contrarrecíproca"),
157     ("(p & q) | ~r", "Proposición con 3 variables"),
158 ]
159
160 def menu():
161     print("\n" + "=" * 55)
162     print(" CLASIFICADOR DE PROPOSICIONES COMPUESTAS")
163     print(" Matemática y Programación I — UTN 2026")
164     print("=" * 55)
165     print(" Operadores: ~p p & q p | q p -> q p <-> q")
166     print(" Variables: cualquier letra (p, q, r, s, ...)")
167     print("=" * 55)
168
169     while True:
170         print()
171         print(" 1. Analizar una proposición propia")
172         print(" 2. Ver ejemplos predefinidos")
173         print(" 3. Salir")
174         opcion = input("\n Opción (1-3): ").strip()
175
176         if opcion == "1":
177             proposicion = input("\n Ingresar proposición: ").strip()
178             if not proposicion:
179                 print(" La proposición no puede estar vacía.")
180                 continue
181             try:
182                 variables, filas = generar_tabla(proposicion)
183                 mostrar_tabla(proposicion, variables, filas, clasificar(filas))
184             except ValueError as e:
185                 print(f"\n Error: {e}\n")
186
187         elif opcion == "2":
188             print()
189             for i, (prop, desc) in enumerate(EJEMPLOS, 1):
190                 print(f" {i}. {prop:<38} ({desc})")
191             try:
192                 sel = int(input("\n Seleccionar número: ").strip())
193                 if 1 <= sel <= len(EJEMPLOS):
194                     proposicion = EJEMPLOS[sel - 1][0]
195                     variables, filas = generar_tabla(proposicion)
196                     mostrar_tabla(proposicion, variables, filas, clasificar(filas))
197             else:
198                 print(" Número fuera de rango.")
```

```
199         except ValueError:
200             print("  Ingresá un número válido.")
201
202     elif opcion == "3":
203         print("\n  ¡Hasta luego!\n")
204         break
205     else:
206         print("  Opción no válida.")
207
208
209 if __name__ == "__main__":
210     menu()
211
```